UNIT 2

Programming in C

Decision Making and Control Flow

Concise exam-oriented notes, important C programs, quick revision, and solved questions

Prepared from the uploaded handwritten Unit 2 notes. A few handwritten details are difficult to read, so the document focuses on clearly identifiable topics and examples.

Contents

• 1. Decision-making and control statements

• 2. if, if-else, nested if, else-if ladder

• 3. switch, conditional operator and goto

• 4. Loops and control-flow programs visible in the notes

• 5. Important programs with code and output

• 6. Quick revision sheet

• 7. Important questions with solutions

1. Decision-Making and Control Statements

A decision-making statement allows a program to choose which statement or block should execute according to a condition. In C, a condition is evaluated as true (non-zero) or false (zero).

Control statements identified in the notes: if statement, switch statement, conditional operator, and goto statement.

Relational operators: <, >, <=, >=, ==, !=.

Logical operators: && (AND), || (OR), ! (NOT).

Important: Use == to compare values. A single = assigns a value.

2. if Statement

The if statement executes a block only when its condition is true.

```
if (condition)
{
    statement;
}
```

Example: if (age >= 18) printf("Eligible");

if-else Statement

The if block runs when the condition is true; otherwise, the else block runs.

```
if (condition)
{
    statements;
}
else
{
    statements;
}
```

Nested if

An if statement placed inside another if or else block is called a nested if. It is useful when one decision depends on another.

```
if (condition1)
{
    if (condition2)
        statement;
    else
        statement;
}
else
{
    statement;
}
```

else-if Ladder

An else-if ladder checks conditions in sequence. The first true block executes; if none is true, the final else block executes.

```
if (condition1)
    statement1;
else if (condition2)
    statement2;
else if (condition3)
    statement3;
else
    default_statement;
```

3. switch Statement

The switch statement selects one case from several alternatives based on an integral expression such as int or char.

```
switch (expression)
{
    case constant1:
        statements;
        break;
    case constant2:
        statements;
        break;
    default:
        statements;
}
```

• **case:** a constant value matched against the expression.

• **break:** exits the switch. Without break, execution may continue into the next case (fall-through).

• **default:** runs when no case matches; it is optional.

• Case labels must be constant integral expressions. Duplicate case values are not allowed.

Conditional (Ternary) Operator

The conditional operator ?: is a compact alternative to a simple if-else.

```
condition ? expression_if_true : expression_if_false;
```

```
int max = (a > b) ? a : b;
```

goto Statement

goto transfers control to a labelled statement in the same function. It can make program flow harder to follow, so loops are usually preferred.

```
goto label;
/* statements */
label:
    statement;
```

4. Loops and Control Flow

The later pages include loop-based examples such as factorial, sum of natural numbers, and Fibonacci series. A loop repeats a block while its condition permits.

for loop

```
for (initialization; condition; update)
{
    statements;
}
```

while loop

```
while (condition)
{
    statements;
}
```

The condition is checked before each iteration. The loop may execute zero times.

do-while loop

```
do
{
```

```
    statements;
} while (condition);
```

The body executes at least once because the condition is checked after the body. Remember the semicolon after while(condition).

break and continue

• **break:** terminates the nearest loop or switch.

• **continue:** skips the remaining statements in the current loop iteration and proceeds to the next iteration.

5. Important C Programs

Use these as exam templates. Each is a complete C program; enter the sample input when prompted.

Program 1: Check positive, negative, or zero

Aim: Classify an entered integer.

```
#include <stdio.h>
int main(void)
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n > 0)
        printf("Positive");
    else if (n < 0)
        printf("Negative");
    else
        printf("Zero");

    return 0;
}
```

Sample output: Input: -4
Output: Negative

Logic: Compare n with zero using an else-if ladder.

Program 2: Check vowel or consonant

Aim: Determine whether an alphabet character is a vowel.

```
#include <stdio.h>
int main(void)
{
    char ch;
    printf("Enter an alphabet: ");
    scanf(" %c", &ch);

    if (ch=='a' || ch=='e' || ch=='i' ||
        ch=='o' || ch=='u' || ch=='A' ||
        ch=='E' || ch=='I' || ch=='O' || ch=='U')
        printf("Vowel");
    else
        printf("Consonant");

    return 0;
}
```

Sample output: Input: E
Output: Vowel

Logic: The leading space in " %c" skips whitespace before reading the character. This version assumes the input is an alphabet.

Program 3: Largest of three numbers

Aim: Find the largest among three integers.

```
#include <stdio.h>
int main(void)
{
    int a, b, c;
    printf("Enter three numbers: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a >= b && a >= c)
        printf("Largest = %d", a);
    else if (b >= a && b >= c)
        printf("Largest = %d", b);
    else
        printf("Largest = %d", c);

    return 0;
}
```

Sample output: Input: 8 12 5
Output: Largest = 12

Logic: && means both comparisons must be true.

Program 4: Smallest of four numbers

Aim: Find the smallest among four integers.

```
#include <stdio.h>
int main(void)
{
    int a, b, c, d, min;
    scanf("%d %d %d %d", &a, &b, &c, &d);
    min = a;

    if (b < min) min = b;
    if (c < min) min = c;
    if (d < min) min = d;

    printf("Smallest = %d", min);
    return 0;
}
```

Sample output: Input: 8 2 5 10
Output: Smallest = 2

Logic: Assume the first value is the minimum, then compare the remaining values.

Program 5: Grade using percentage

Aim: Display a grade according to percentage. The thresholds below are an illustrative example; use the exact thresholds given by your teacher.

```
#include <stdio.h>
int main(void)
{
    float p;
    printf("Enter percentage: ");
    scanf("%f", &p);

    if (p < 0 || p > 100)
        printf("Invalid percentage");
    else if (p >= 90)
        printf("Grade A");
    else if (p >= 75)
        printf("Grade B");
    else if (p >= 60)
        printf("Grade C");
    else if (p >= 40)
        printf("Grade D");
    else
        printf("Fail");

    return 0;
}
```

Sample output: Input: 82
Output: Grade B

Logic: Check higher ranges first so the first matching condition is selected.

Program 6: Simple calculator using switch

Aim: Choose an arithmetic operation using an operator.

```
#include <stdio.h>
int main(void)
{
    double a, b;
    char op;
    printf("Enter: number operator number: ");
    scanf("%lf %c %lf", &a, &op, &b);

    switch (op)
    {
        case '+': printf("= %.2f", a + b); break;
```

```
        case '-': printf("= %.2f", a - b); break;
        case '*': printf("= %.2f", a * b); break;
        case '/':
            if (b != 0)
                printf("= %.2f", a / b);
            else
                printf("Cannot divide by zero");
            break;
        default: printf("Invalid operator");
    }
    return 0;
}
```

Sample output: Input: 9 * 3
Output: = 27.00

Logic: Each case handles one operator; break prevents fall-through.

Program 7: Factorial using a loop

Aim: Calculate n! for a non-negative integer.

```
#include <stdio.h>
int main(void)
{
    int n, i;
    unsigned long long fact = 1;
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);

    if (n < 0)
        printf("Invalid input");
    else if (n > 20)
        printf("Number too large for this data type");
    else {
        for (i = 1; i <= n; i++)
            fact = fact * i;
        printf("Factorial = %llu", fact);
    }
    return 0;
}
```

Sample output: Input: 5
Output: Factorial = 120

Logic: Start fact at 1 and multiply it by every integer from 1 to n. 0! = 1.

Program 8: Sum of first n natural numbers

Aim: Calculate 1 + 2 + ... + n.

```
#include <stdio.h>
int main(void)
{
    int n, i, sum = 0;
    printf("Enter n: ");
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
        sum += i;

    printf("Sum = %d", sum);
    return 0;
}
```

Sample output: Input: 5
Output: Sum = 15

Logic: sum += i means sum = sum + i.

Program 9: Fibonacci series

Aim: Print the first n Fibonacci terms (starting 0, 1).

```
#include <stdio.h>
int main(void)
{
    int n, i;
```

```
    long long a = 0, b = 1, next;
    printf("Number of terms: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("%lld ", a);
        next = a + b;
        a = b;
        b = next;
    }
    return 0;
}
```

Sample output: Input: 7
Output: 0 1 1 2 3 5 8

Logic: Each new term is the sum of the previous two terms.

Program 10: Check even or odd using conditional operator

Aim: Use ?: to choose one of two messages.

```
#include <stdio.h>
int main(void)
{
    int n;
    scanf("%d", &n);
    (n % 2 == 0) ? printf("Even") : printf("Odd");
    return 0;
}
```

Sample output: Input: 11
Output: Odd

Logic: The remainder operator % returns the remainder after integer division.

6. Quick Revision Sheet

Concept	Remember
if	Runs block when condition is true.
if-else	Selects between two alternatives.
Nested if	An if statement inside another if/else.
else-if ladder	Checks conditions from top to bottom.
switch	Selects a case based on an integral expression.
break	Exits the nearest loop or switch.
default	Runs if no switch case matches.
?:	Conditional/ternary operator: condition ? true : false.
goto	Transfers control to a labelled statement; use sparingly.
for	Best when the iteration pattern/count is known.
while	Checks condition before each iteration.
do-while	Runs body at least once; semicolon after condition.
&& / || / !	Logical AND / OR / NOT.
== / =	Equality comparison / assignment.

Common mistakes

• Writing if (a = b) when comparison was intended; use if (a == b).

• Forgetting & before variables in scanf(), for example scanf("%d", &n;).

• Forgetting break in switch when fall-through is not intended.

• Using = instead of == inside a condition.

• Forgetting the semicolon at the end of do-while.

• Not initialising accumulators such as sum = 0 or fact = 1.

7. Important Questions with Solutions

The uploaded file is a handwritten notebook scan rather than a typed question sheet. The following questions are based on the clearly identifiable topics and example prompts in the notes; where an exact original wording was unreadable, the question is stated in standard exam language.

Q1. What is a decision-making statement in C?

Decision-making statements control program flow by choosing which statement or block executes according to a condition. Examples include if, if-else, else-if ladder, and switch.

Q2. Write the general syntax of an if-else statement.

if (condition) { /* statements when true */ } else { /* statements when false */ }.

Q3. What is a nested if statement?

A nested if is an if statement written inside another if or else block. It is used when a second decision depends on the first decision.

Q4. Explain an else-if ladder.

It checks several conditions sequentially. The block for the first true condition executes; if all are false, the final else block executes, if present.

Q5. What is the purpose of break in a switch statement?

break terminates the switch statement and transfers control to the statement after it. Without break, execution can continue into following cases.

Q6. Differentiate between if-else and switch.

if-else can test relational and logical expressions and ranges. switch compares one expression with constant case labels and is useful for multiple fixed alternatives.

Q7. What is the conditional operator? Give an example.

The conditional operator ?: is a three-operand operator. Example: max = (a > b) ? a : b; assigns the larger of a and b to max.

Q8. What is goto?

goto transfers program control to a labelled statement in the same function. Excessive use can make code difficult to read; loops are generally preferred.

Q9. Differentiate between while and do-while.

while checks the condition before the body, so it may execute zero times. do-while checks after the body, so it executes at least once.

Q10. Write a C program to find the largest among three numbers.

Use Program 3 above. It compares each number against the other two using an else-if ladder.

Q11. Write a C program to check whether a character is a vowel or consonant.

Use Program 2 above. It compares the character with upper- and lowercase vowel letters.

Q12. Write a C program to find the factorial of a number.

Use Program 7 above. Initialise factorial to 1 and multiply by each integer from 1 through n. Factorial of 0 is 1.

Q13. Write a C program to find the sum of the first n natural numbers.

Use Program 8 above. Initialise sum to zero and add each value of i from 1 through n.

Q14. Write a C program to print a Fibonacci series.

Use Program 9 above. Start with a = 0 and b = 1; print a, then update the two previous terms.

Q15. Explain the use of default in switch.

default is optional and executes when none of the case values match the switch expression.

Q16. What is the difference between break and continue?

break exits the nearest loop or switch. continue skips the rest of the current loop iteration and starts the next iteration.

Programming in C - Unit 2

Decision Making, Control Flow, Loops, Functions and Recursion

Concise Exam Notes | Important Programs | Syntax & Differences | Solved Output Questions

Part	Contents
Part 1 - Theory	Decision making and control statements, structure of a C program, if / if-else / nested-if / else-if ladder, conditional operator, loops (for, while, do-while), break, continue, switch, goto, functions, recursion
Part 2 - Programs	41 complete, compiled and tested C programs with aim, code, sample input/output, logic and points to remember
Part 3 - Syntax & Differences	Syntax cheat-sheet and comparison tables (if vs if-else, while vs do-while, break vs continue, switch vs if-else ladder, predefined vs user-defined function, iteration vs recursion, etc.)
Part 4 - Output Questions	10 solved output-based questions with explanation
Part 5 - Revision	Practice-question index, viva Q&A, common mistakes and last-minute checklist

PART 1 - THEORY

1. Decision Making and Control Statements

- **Decision-making structures** require the programmer to specify one or more conditions to be evaluated. A statement is executed **only if the condition becomes true**.
- C supports decision making through these statements: **(1) if statement, (2) switch statement, (3) conditional operator (?:), (4) goto statement**.
- These are called **decision-making statements** or **control statements**.

Category	Statements	Purpose
Conditional (selection)	if, if-else, nested if, else-if ladder	Choose one path depending on a condition
Looping (iteration)	for, while, do-while	Repeat a block of code until a condition is false
Jump	switch, goto (also break, continue)	Transfer control to another part of the program

2. Structure of a C Program

Section	What it contains
(i) Documentation	Comments: program name, author, purpose. /* ... */ or // ...
(ii) Preprocessor (link section)	Header files, e.g. `#include <stdio.h>`
(iii) Definition section	Symbolic constants, e.g. `#define PI 3.14`
(iv) Global declaration	Global variables (usable in all functions) and function prototypes
(v) main() function	Execution starts here. Contains local variables, statements, function calls and `return 0;`
(vi) Sub-programs	User-defined functions (usually written below main)

```
/* Documentation: program to add two numbers */
#include <stdio.h>              /* preprocessor / header file   */
#define LIMIT 100               /* definition section           */
int total = 0;                  /* global variable              */
int add(int, int);              /* function prototype           */

int main()                      /* main function                */
{
    int x = 5, y = 7;           /* local variables              */
    total = add(x, y);          /* statement + function call    */
    printf("%d\n", total);
    return 0;
}

int add(int a, int b)           /* sub program (user-defined)   */
{
    return a + b;
}
```

3. if Statement

- C uses the keyword **if** to implement decision control. The block runs only when the test condition is **true**; if it is false the block is skipped and the next statement executes.

```
if (test condition)
{
    block statement;        /* executes only if condition is TRUE */
}
next statement;
```

Flow chart:

```
       Start
         |
         v
  < condition test? > ---- False ----+
         | True                      |
         v                           |
  [ if block executes ]              |
         |                           |
         +<--------------------------+
         |
         v
        Exit
```

- Example: `if (age >= 18) printf("eligible");` (Programs 1 and 5 show more examples).

4. if-else Statement

- The if statement alone does something only when the expression is true and **does nothing when it is false**.
- To do something when the condition is false we use the **else** part. So if-else executes **one of two blocks**.

```
if (test condition)
{
    Statement 1;            /* condition true  */
}
else
{
    Statement 2;            /* condition false */
}
```

```
        Start
          |
          v
   < condition check? >
     |            |
   True         False
     |            |
     v            v
 [if block]   [else block]
     |            |
     +-----+------+
           |
           v
          Exit
```

5. Nested if / Nested if-else

- It is **always allowed** to define an if or if-else statement inside another if-block or else-block. This is called nesting.
- Use it when a second decision depends on the result of the first (e.g. largest of three numbers, Program 12).

```
if (C1)
{
    if (C2)
        { St1; }
    else
        { St2; }
}
else
{
    if (C3)
        { St3; }
    else
        { St4; }
}
```

- Flow: test C1; if true test C2 (St1 or St2); if C1 is false test C3 (St3 or St4).

6. else-if Ladder (multiple conditions)

- Used to test a **set of conditions in sequence**. If any condition is true, the corresponding block executes and control **exits the whole ladder**. If all conditions are false, the final **else** block executes.
- Conditions are checked from top to bottom, so the order of conditions matters.

```
if (C1)
{   block 1;   }
else if (C2)
{   block 2;   }
else if (C3)
{   block 3;   }
else
{   default block;   }
```



7. Conditional (Ternary) Operator ?:

- Short form of if-else. Syntax: `variable = (condition) ? expression1 : expression2;`
- If the condition is true, expression1 is used, otherwise expression2. Example: `max = (a > b) ? a : b;` (Program 15).

8. Loops

- In programming, a **loop** is used to **repeat a block of code until the specified condition is met**. C has three loops: **for, while, do-while**.

8.1 for loop

```
for (initialization; test expression; update)
{
    statements;          /* body of the loop */
}
```

How the for loop works:

- The **initialization** statement is executed **once**.
- Then the **test expression** is evaluated. If it is **false**, the for loop is terminated.
- If the test expression is **true**, the statements inside the body are executed and the **update** expression is updated.
- Again the test expression is evaluated. The process goes on until the test expression is false; then the loop terminates.

```
 Start
   |
   v
 [Initialization]
   |
   v
 < condition? > ---- False ----> Exit
   | True
   v
 [Statements] --> [Update] --> (go back to condition)
```

```
for (a = 1; a <= 10; a++)
    printf("a = %d\n", a);        /* prints 1 to 10 */
```

8.2 while loop

```
while (test expression)
{
    statements;
}
```

How the while loop works:

- The while loop evaluates the **test expression inside the parentheses**.
- If the test expression is **true**, the statements inside the body are executed. Then the test expression is evaluated again.
- The process goes on until the test expression is evaluated to **false**.
- If the test expression is false, the loop terminates (ends).

```
 Start
   |
   v
 < C? > ---- False ----> Exit
   | True
   v
 [Body of while] ---> (go back to C?)
```

8.3 do-while loop

- Similar to the while loop with one **important difference**: the body of do-while is **executed at least once**. Only then the test expression is evaluated.

```
do
{
    statements;
}
while (test expression);       /* semicolon is compulsory */
```

How the do-while loop works:

- The body of the do-while loop is executed **once only**. Then the test expression is evaluated.
- If the test expression is true, the body of the loop is executed again and the test expression is evaluated once more.
- This process goes on until the test expression becomes false. If it is false, the loop ends.

```
 Start
   |
   v
 [Body of do] <-------+
   |                  |
   v                  |
 < C? > ---- True ----+
   | False
   v
  Exit
```

8.4 break statement

- The **break** statement **ends the loop immediately** when it is encountered. Syntax: `break;`
- It is always used with an **if** statement inside the loop, e.g. `if (i == 7) break;`
- break is used only in **loops and switch**.

8.5 continue statement

- The **continue** statement **skips the current iteration** of the loop and continues with the next iteration. Syntax: `continue;`, e.g. `if (i == 7) continue;`

9. switch Statement

- switch provides a **multi-way branch**. It allows selecting one of several alternatives depending upon the **value of an expression**. The expression is enclosed within parentheses.
- Control is transferred to the matching **case label**, and the statements after that label are executed.
- The expression must be of type **int or char only**; case labels must be unique constants.
- If **break** is not used, all statements after the matching label are also executed (**fall-through**).
- The **default** case is optional; it runs when no case matches.

```
switch (expression / value)
{
    case constant1:
        // Statement 1
        break;
    case constant2:
        // Statement 2
        break;
    ...
    default:
        // default statement
}
```

```
 Start --> [Switch expression]
               |
 Case 1? --True--> Statement 1 --+
   | False                       |
 Case 2? --True--> Statement 2 --+
   | False                       |
 Case n? --True--> Statement 3 --+--> Exit
   | False                       |
 Default ---------> Statement 4 -+
```

10. goto Statement

- goto is a control-flow statement used to make an **unconditional jump** from the goto line to a **labelled statement within the same function**.
- Mainly used for **error handling** or **exiting deeply nested loops**.

```
goto label;
    // statements
label:
    // statement to jump to
```

- Example: Program 25 (even / odd using goto).

11. Functions

- A **function** is a set of statements that takes input, does some specific computation and produces output. The idea is to put commonly or repeatedly used tasks together in a function so that, instead of writing the same code again and again, we call the function with different inputs.
- A C function contains a set of instructions enclosed by **{ }** which perform a specific operation. It divides a complex problem into smaller parts and makes the program easy to understand and reuse.
- Every C program has at least one function, which is **main()**.

Why do we need functions?

- **Readability** - improves the readability of code.
- **Reusability** - the same function can be used in any program.
- **Easy debugging** - errors are easy to trace.
- **Smaller code** - duplicate sets of statements are replaced by a function call.

Types of functions

Point	Predefined / Library function	User-defined function
Meaning	Ready-made function provided by the compiler library; used directly	Function created by the user for a specific task
Header file	Header file must be included	Not required
Code	No need to write code; already defined in the library	User writes the code
Examples	`printf()`, `scanf()` (stdio.h), `sqrt()` (math.h)	addNumbers(), factorial()

- `printf()` sends formatted output to the screen; defined in `<stdio.h>` so include it using `#include`.
- `sqrt()` calculates the square root of a number; defined in `<math.h>`.

User-defined function - 3 components

Component	Meaning	Example
1. Function declaration / prototype	Tells the compiler the function name, return type and parameter types before it is used. Placed at the top, above main(). (If the whole function is defined above main, the declaration becomes optional.)	`int add(int, int);`
2. Function definition	Contains the actual block of code that executes the logic when the function is triggered. Usually written below main().	`int add(int a, int b) { return a + b; }`
3. Function call	How you actually run the function from main() or another function. Calling a function pauses the current code, transfers control to the user-defined function, then returns.	`sum = add(x, y);`

```
#include <stdio.h>
// 1. function declaration
return_type function_name(datatype);

int main()
{
    // function call
    function_name(argument);
    return 0;
}

// 2. function definition
return_type function_name(datatype parameter)
{
    // statements
    return value;
}
```

Key parts of a function:

- **Return type** - the type of data the function sends back to the main program: int, float, char, etc. Use the **void** keyword if it returns nothing.
- **Return statement** - uses the keyword **return** to output the final result.
- **Arguments** are the values passed in the call; **parameters** are the variables that receive them in the definition.

12. Recursion

- **Recursion** is a technique where a **function calls itself** to solve smaller parts of a problem until it reaches a stopping condition. Instead of repeating code with a loop, recursion breaks the problem down into simpler versions of the same problem.
- It continues calling itself with new values until a specific condition is met, known as the **base case**.

```
return_type function_name(dt P1, dt P2, ...)
{
    if (base condition)
        return value;                   // base case
    return function_name(P1, P2);       // recursive case
}
```

Term	Meaning
Base case	The condition where recursion **stops**; it avoids an infinite call.
Recursive case	The part where the function **keeps calling itself** with a smaller or simpler input.

How does recursion work? (call stack)

- When a function call is made it is added (**pushed**) to a **call stack** - a special memory structure that keeps track of active function calls.
- The function keeps calling itself and each new call is pushed on top of the stack.
- Recursion is **discontinued** when the base case is reached and no more recursive calls are made.
- Then a value is returned and the function calls start **resolving one by one from the top of the stack to the bottom**.

Types of recursion

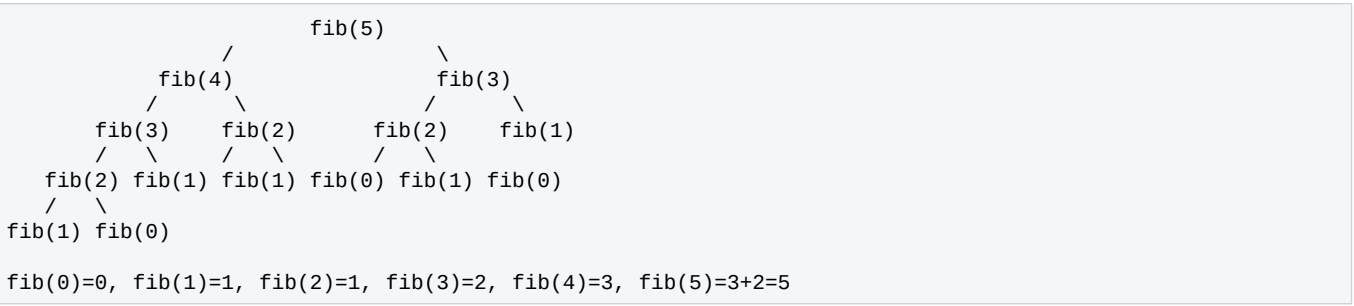
Type	Meaning	Example
Direct	A function calls **itself** directly	fact(n) calls fact(n-1)
Indirect	Function A calls function B and B calls A back	funA() calls funB(), funB() calls funA() (Program 41)
Tree	A function makes **more than one** recursive call, forming a tree of calls	fib(n) = fib(n-1) + fib(n-2)

Dry run 1: factorial using recursion - fact(4)

Call	Expression waiting	Value returned
fact(4)	4 * fact(3)	4 x 6 = 24
fact(3)	3 * fact(2)	3 x 2 = 6
fact(2)	2 * fact(1)	2 x 1 = 2
fact(1)	1 * fact(0)	1 x 1 = 1
fact(0)	BASE CASE	1

- Calls go down (4, 3, 2, 1, 0); returns come back up (1, 1, 2, 6, 24). So fact(4) = 24.

Dry run 2: Fibonacci using recursion - fib(5) call tree (tree recursion)



Dry run 3: GCD using recursion - gcd(48, 18)

Call	a % b	Next call / result
gcd(48, 18)	48 % 18 = 12	calls gcd(18, 12)
gcd(18, 12)	18 % 12 = 6	calls gcd(12, 6)
gcd(12, 6)	12 % 6 = 0	calls gcd(6, 0)
gcd(6, 0)	b == 0	BASE CASE - returns 6; 6 comes back through all calls. GCD = 6

PART 2 - IMPORTANT C PROGRAMS (all compiled and tested)

Each program shows: Aim, Code, Sample Input/Output, Logic and Remember. In the output box, the value typed by the user appears after the prompt.

A. if, if-else, nested-if and else-if programs

Program 1: Accept a number and print if it is odd

Aim: To use a simple if statement (no else part).

```
#include <stdio.h>

int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    if (num % 2 != 0)
        printf("The number is odd.\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a number: 7
The number is odd.
```

Logic:

- An odd number leaves remainder 1 when divided by 2, so num % 2 != 0 is true.
- If the condition is false (even number) nothing is printed.

Remember:

- **No semicolon after if (...)**
- **% is the modulus (remainder) operator**
- **&num in scanf**

Program 2: Ratio of two floating numbers - exchange if ratio > 0

Aim: To accept two float numbers, compute a/b and swap them when the ratio is greater than 0.

```
#include <stdio.h>

int main()
{
    float a, b, temp, ratio;
    printf("Enter two floating-point numbers: ");
    scanf("%f %f", &a, &b);

    if (b == 0)
    {
        printf("Ratio not possible (b is 0)\n");
        return 0;
    }

    ratio = a / b;
    printf("Ratio = %.2f\n", ratio);

    if (ratio > 0)
    {
        temp = a;          /* swap using a third variable */
        a = b;
        b = temp;
        printf("Numbers after exchange:\n");
        printf("a = %.2f\n", a);
        printf("b = %.2f\n", b);
    }
    return 0;
}
```

Sample Input / Output:

```
Enter two floating-point numbers: 6 3
Ratio = 2.00
Numbers after exchange:
a = 3.00
b = 6.00
```

Logic:

- ratio = a / b is stored in a float variable.
- If ratio > 0, the values of a and b are swapped with temp (temp = a; a = b; b = temp).
- The check b == 0 avoids division by zero (extra safety).

Remember:

- **Format specifier for float is %f**
- **Three-step swap with temp**

Program 3: Check whether two numbers are equal or unequal

Aim: To use if-else for a two-way decision.

```
#include <stdio.h>

int main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a == b)
        printf("Numbers are equal\n");
    else
        printf("Numbers are unequal\n");

    return 0;
}
```

Sample Input / Output:

```
Enter two numbers: 15 20
Numbers are unequal
```

Logic:

- == compares two values; if true the if-part runs, otherwise the else-part runs.

Remember:

- **Use == for comparison, = is assignment**

Program 4: Accept a character - print Yes if it is 'y', otherwise No

Aim: To compare a character using if-else.

```
#include <stdio.h>

int main()
{
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);      /* space before %c skips old newline */

    if (ch == 'y')
        printf("Yes\n");
    else
        printf("No\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a character: y
Yes
```

Logic:

- A character constant is written in single quotes: 'y'.
- ch == 'y' is true only for lower-case y.

Remember:

- **%c for char**

- **single quotes for char, double quotes for strings**

Program 5: Check whether a number is odd or even

Aim: To use if-else with the modulus operator.

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 2 == 0)
        printf("Even number\n");
    else
        printf("Odd number\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a number: 10
Even number
```

Logic:

- If n is divisible by 2 (remainder 0) it is even, otherwise odd.

Remember:

- **n % 2 == 0 -> even**

Program 6: Find the larger and smaller of two numbers

Aim: To use an if - else if - else chain (also handles equal numbers).

```
#include <stdio.h>

int main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        printf("%d is larger and %d is smaller\n", a, b);
    else if (b > a)
        printf("%d is larger and %d is smaller\n", b, a);
    else
        printf("Both numbers are equal\n");

    return 0;
}
```

Sample Input / Output:

```
Enter two numbers: 12 30
30 is larger and 12 is smaller
```

Logic:

- First test a > b, then b > a; if both fail the numbers must be equal.

Remember:

- **else part handles the remaining case**

Program 7: Check whether a number is positive, negative or zero

Aim: To use the else-if ladder with three outcomes.

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n > 0)
        printf("Positive number\n");
    else if (n < 0)
        printf("Negative number\n");
    else
        printf("The number is zero\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a number: -8
Negative number
```

Logic:

- n > 0 -> positive, n < 0 -> negative, otherwise the number is 0.

Remember:

- **Zero is neither positive nor negative**

Program 8: Check whether a number is divisible by 5

Aim: To test divisibility using %.

```
#include <stdio.h>

int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n % 5 == 0)
        printf("Number is divisible by 5\n");
    else
        printf("Number is not divisible by 5\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a number: 35
Number is divisible by 5
```

Logic:

- A number is divisible by 5 when the remainder after dividing by 5 is 0.

Remember:

- **n % k == 0 means n is divisible by k**

Program 9: Check whether a year is a leap year

Aim: To use logical operators (&&, ||) inside an if condition.

```
#include <stdio.h>

int main()
{
    int year;
    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0))
        printf("%d is a leap year\n", year);
    else
        printf("%d is not a leap year\n", year);

    return 0;
}
```

Sample Input / Output:

```
Enter a year: 2024
2024 is a leap year
```

Logic:

- Leap year rule: divisible by 4 but not by 100, OR divisible by 400.
- Examples: 2024 (leap), 1900 (not leap), 2000 (leap).

Remember:

- **&& = AND, || = OR**
- **Remember the rule: (4 and not 100) or 400**

Program 10: Check whether a character is an upper-case or lower-case alphabet

Aim: To compare characters using their ASCII ranges.

```
#include <stdio.h>

int main()
{
    char ch;
    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (ch >= 'A' && ch <= 'Z')
        printf("Upper case alphabet\n");
    else if (ch >= 'a' && ch <= 'z')
        printf("Lower case alphabet\n");
    else
        printf("Not an alphabet\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a character: G
Upper case alphabet
```

Logic:

- Characters are stored as ASCII codes: 'A'-'Z' = 65-90, 'a'-'z' = 97-122.
- Range check with && decides the case.

Remember:

- **Both conditions joined with &&**

Program 11: Input an alphabet and check whether it is a vowel or a consonant

Aim: To use a nested if (alphabet test first, vowel test inside).

```
#include <stdio.h>

int main()
{
    char ch;
    printf("Enter an alphabet: ");
    scanf(" %c", &ch);

    if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z'))
    {
        if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
            ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U')
            printf("%c is a vowel\n", ch);
        else
            printf("%c is a consonant\n", ch);
    }
    else
        printf("Not an alphabet\n");

    return 0;
}
```

Sample Input / Output:

```
Enter an alphabet: e
e is a vowel
```

Logic:

- Outer if checks that the character is a letter.
- Inner if checks the 5 vowels in both cases; every other letter is a consonant.

Remember:

- **Nested if = if inside an if-block**

Program 12: Find the largest among 3 numbers (nested if-else)

Aim: To understand nested if-else.

```
#include <stdio.h>

int main()
{
    int a, b, c;
    printf("Enter the value of a, b, c: ");
    scanf("%d %d %d", &a, &b, &c);

    if (a > b)
    {
        if (a > c)
            printf("a is greatest no.\n");
        else
            printf("c is greatest no.\n");
    }
    else
    {
        if (b > c)
            printf("b is greatest no.\n");
        else
            printf("c is greatest no.\n");
    }
    return 0;
}
```

Sample Input / Output:

```
Enter the value of a, b, c: 10 25 17
b is greatest no.
```

Logic:

- If a > b, the winner is a or c. Otherwise the winner is b or c.
- Only two comparisons are executed for any input.

Remember:

- **Trace: a>b true -> a>c ? a : c**

- **Braces keep each else with the correct if**

Program 13: Largest and smallest among 4 numbers

Aim: To find the maximum and minimum with comparison statements (for 3 numbers just remove d).

```
#include <stdio.h>

int main()
{
    int a, b, c, d, large, small;
    printf("Enter four numbers: ");
    scanf("%d %d %d %d", &a, &b, &c, &d);

    large = a;              /* assume first number is both */
    small = a;

    if (b > large) large = b;
    if (c > large) large = c;
    if (d > large) large = d;

    if (b < small) small = b;
    if (c < small) small = c;
    if (d < small) small = d;

    printf("Largest = %d\n", large);
    printf("Smallest = %d\n", small);
    return 0;
}
```

Sample Input / Output:

```
Enter four numbers: 12 45 7 30
Largest = 45
Smallest = 7
```

Logic:

- Assume the first number is the largest and smallest.
- Compare every other number with the current largest / smallest and update if needed.

Remember:

- **Initialise large and small with a first value**

Program 14: Student result / grade (else-if ladder)

Aim: To prepare the result of a student from marks of 5 subjects (English, Hindi, Maths, Science, Social Science) out of 100 each, print marks obtained, percentage and grade.

```
#include <stdio.h>

int main()
{
    int english, hindi, maths, science, ss, total;
    float percentage;

    printf("Enter marks of 5 subjects: ");
    scanf("%d %d %d %d %d", &english, &hindi, &maths, &science, &ss);

    total = english + hindi + maths + science + ss;
    percentage = total / 500.0 * 100;

    printf("Obtained marks = %d\n", total);
    printf("Percentage = %.2f\n", percentage);

    if (percentage >= 90)
        printf("Grade A\n");
    else if (percentage >= 80)
        printf("Grade B\n");
    else if (percentage >= 70)
        printf("Grade C\n");
    else if (percentage >= 60)
        printf("Grade D\n");
    else
        printf("Grade E\n");

    return 0;
}
```

Sample Input / Output:

```
Enter marks of 5 subjects: 85 90 78 88 92
Obtained marks = 433
Percentage = 86.60
Grade B
```

Logic:

- Grade table: A = 90% and above, B = 80-90, C = 70-80, D = 60-70, E = below 60.
- Conditions are tested from the top; the first true condition decides the grade, so the lower limit is enough.
- Divide by 500.0 (not 500) so the result is a float.

Remember:

- **500.0 avoids integer division**
- **Order of the ladder matters**

B. Conditional operator

Program 15: Larger of two numbers using the conditional (?:) operator

Aim: To replace a simple if-else by the conditional operator.

```
#include <stdio.h>

int main()
{
    int a, b, max;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    max = (a > b) ? a : b;      /* condition ? value_if_true : value_if_false */

    printf("Larger number = %d\n", max);
    return 0;
}
```

Sample Input / Output:

```
Enter two numbers: 10 25
Larger number = 25
```

Logic:

- If (a > b) is true the value after ? is used, otherwise the value after : is used.

Remember:

- **Syntax: variable = (condition) ? expr1 : expr2;**

C. Loop programs (for, while, do-while, break, continue)

Program 16: Print numbers 1 to 10 using for loop

Aim: To understand the working of the for loop.

```
#include <stdio.h>

int main()
{
    int a;
    for (a = 1; a <= 10; a++)
        printf("%d ", a);

    printf("\n");
    return 0;
}
```

Output:

```
1 2 3 4 5 6 7 8 9 10
```

Logic:

- Initialisation a = 1 runs once; condition a <= 10 is tested before every pass; a++ runs after every pass.
- Loop stops when a becomes 11 (condition false).

Remember:

- **for (initialisation; condition; update)**

Program 17: Sum of the first 10 natural numbers (for loop)

Aim: To accumulate a sum inside a loop.

```
#include <stdio.h>

int main()
{
    int i, sum = 0;

    for (i = 1; i <= 10; i++)
        sum = sum + i;

    printf("Sum of first 10 natural numbers = %d\n", sum);
    return 0;
}
```

Output:

```
Sum of first 10 natural numbers = 55
```

Logic:

- sum starts from 0 and i is added in every iteration: 1 + 2 + ... + 10 = 55.
- Formula check: n(n+1)/2 = 10 x 11 / 2 = 55.

Remember:

- **Always initialise sum = 0**

Program 18: Print numbers 1 to 10 using while loop

Aim: To understand the working of the while loop (entry-controlled).

```
#include <stdio.h>

int main()
{
    int a = 1;
    while (a <= 10)
    {
        printf("a = %d\n", a);
        a++;
    }
    return 0;
}
```

Output:

```
a = 1
a = 2
a = 3
a = 4
a = 5
a = 6
a = 7
a = 8
a = 9
a = 10
```

Logic:

- The test expression is evaluated first. While it is true the body runs, then the test is evaluated again.
- a++ inside the body prevents an infinite loop.

Remember:

- **Do not forget the update (a++)**

Program 19: Print numbers 1 to 10 using do-while loop

Aim: To understand the do-while loop (exit-controlled).

```
#include <stdio.h>

int main()
{
    int i = 1;
    do
    {
        printf("i = %d\n", i);
        i++;
    }
    while (i <= 10);          /* semicolon is compulsory here */

    return 0;
}
```

Output:

```
i = 1
i = 2
i = 3
i = 4
i = 5
i = 6
i = 7
i = 8
i = 9
i = 10
```

Logic:

- The body runs once first, and only then the test expression is evaluated.
- If the test is true the body runs again; it stops when the test is false.

Remember:

- **do { ... } while (condition); <- semicolon**

Program 20: Difference demo: while vs do-while when the condition is false at start

Aim: To show that do-while executes at least once.

```
#include <stdio.h>

int main()
{
    int i = 11;

    while (i <= 10)
        printf("while: i = %d\n", i);      /* never executes */

    do
    {
        printf("do-while: i = %d\n", i);   /* executes once */
    }
    while (i <= 10);

    return 0;
}
```

Output:

```
do-while: i = 11
```

Logic:

- Since 11 <= 10 is false at the beginning, the while body is skipped.
- The do-while body executes once before the test.

Remember:

- **do-while = at least one execution**

Program 21: Sum of numbers entered by the user - stop when a negative number is entered (break)

Aim: Input 10 numbers and add them. If the user enters a negative number the loop terminates.

```
#include <stdio.h>

int main()
{
    int i, num, sum = 0;

    for (i = 1; i <= 10; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &num);

        if (num < 0)
            break;              /* leave the loop immediately */

        sum = sum + num;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Sample Input / Output:

```
Enter number 1: 5
Enter number 2: 3
Enter number 3: -1
Sum = 8
```

Logic:

- break ends the loop at once when the condition num < 0 is true.
- Control goes to the first statement after the loop (the printf of Sum).

Remember:

- **break is used with if inside a loop (or inside switch)**

Program 22: Sum of numbers - negative numbers are skipped (continue)

Aim: Input 10 numbers; a negative number is not added to the sum.

```
#include <stdio.h>

int main()
{
    int i, num, sum = 0;

    for (i = 1; i <= 10; i++)
    {
        printf("Enter number %d: ", i);
        scanf("%d", &num);

        if (num < 0)
            continue;           /* skip the rest of this iteration */

        sum = sum + num;
    }
    printf("Sum = %d\n", sum);
    return 0;
}
```

Sample Input / Output:

```
Enter number 1: 1
Enter number 2: -2
Enter number 3: 3
Enter number 4: 4
Enter number 5: 5
Enter number 6: 6
Enter number 7: 7
Enter number 8: 8
Enter number 9: 9
Enter number 10: 10
Sum = 53
```

Logic:

- continue skips the remaining statements of the current iteration and goes to the next iteration.

- -2 is ignored: 1+3+4+5+6+7+8+9+10 = 53.

Remember:

- **continue does not stop the loop, break does**

D. switch and goto programs

Program 23: Simple switch statement

Aim: To understand switch, case, break and default.

```
#include <stdio.h>

int main()
{
    int a = 3;

    switch (a)
    {
        case 1:
            printf("one\n");
            break;
        case 2:
            printf("two\n");
            break;
        case 3:
            printf("three\n");
            break;
        default:
            printf("Null\n");
    }
    return 0;
}
```

Output:

```
three
```

Logic:

- The value of a (3) is matched with each case label; case 3 matches, so its statement runs.
- break leaves the switch. default runs when no case matches.

Remember:

- **case label ends with a colon (:)**
- **break after every case**

Program 24: Simple calculator using switch

Aim: To select an operation (+, -, *, /, %) using switch on a char.

```c
#include <stdio.h>

int main()
{
    char op;
    float a, b;

    printf("Enter any operation (+, -, *, /, %%): ");
    scanf(" %c", &op);

    printf("Enter two operands: ");
    scanf("%f %f", &a, &b);

    switch (op)
    {
        case '+':
            printf("%.2f + %.2f = %.2f\n", a, b, a + b);
            break;
        case '-':
            printf("%.2f - %.2f = %.2f\n", a, b, a - b);
            break;
        case '*':
            printf("%.2f * %.2f = %.2f\n", a, b, a * b);
            break;
        case '/':
            if (b != 0)
                printf("%.2f / %.2f = %.2f\n", a, b, a / b);
            else
                printf("Division by zero is not possible\n");
            break;
        case '%':
            if ((int)b != 0)
                printf("%d %% %d = %d\n", (int)a, (int)b, (int)a % (int)b);
            else
                printf("Division by zero is not possible\n");
            break;
        default:
            printf("Invalid operator\n");
    }
    return 0;
}
```

Sample Input / Output:

```
Enter any operation (+, -, *, /, %): *
Enter two operands: 6 4
6.00 * 4.00 = 24.00
```

Logic:

- The switch expression is a char; each case is a character constant such as '+'.
- % works only with integers, so the operands are type-cast to int.
- To print a % sign with printf, write %%.

Remember:

- **switch works with int / char only**
- **Type cast: (int)a**

Program 25: goto statement - check even / odd

Aim: To make an unconditional jump to a labelled statement in the same function.

```
#include <stdio.h>

int main()
{
    int a = 5;

    if (a % 2 == 0)
        goto even;
    else
        goto odd;

even:
    printf("given no. is even\n");
    return 0;

odd:
    printf("given no. is odd\n");
    return 0;
}
```

Output:

```
given no. is odd
```

Logic:

- goto label; transfers control to the statement that follows label: .
- Here a = 5 is odd, so control jumps to the odd label.

Remember:

- **Label name is followed by a colon**
- **Jump only inside the same function**

E. Function programs

Program 26: Add two numbers using a user-defined function

Aim: To learn function prototype, definition and call.

```
#include <stdio.h>

int addNumbers(int a, int b);          /* 1. function declaration (prototype) */

int main()
{
    int n1, n2, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &n1, &n2);

    sum = addNumbers(n1, n2);          /* 2. function call */
    printf("Sum = %d\n", sum);
    return 0;
}

int addNumbers(int a, int b)           /* 3. function definition */
{
    int result;
    result = a + b;
    return result;                     /* value sent back to main() */
}
```

Sample Input / Output:

```
Enter two numbers: 15 20
Sum = 35
```

Logic:

- main() passes n1 and n2 (arguments); the function receives them in a and b (parameters).
- return sends the result back and it is stored in sum.

Remember:

- **Return type, name and parameter types must match in prototype and definition**

Program 27: Find the larger of two numbers using a function

Aim: To return a value from a function using if-else.

```
#include <stdio.h>

int large(int a, int b);

int main()
{
    int a, b, max;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    max = large(a, b);
    printf("Larger number = %d\n", max);
    return 0;
}

int large(int a, int b)
{
    if (a > b)
        return a;
    else
        return b;
}
```

Sample Input / Output:

```
Enter two numbers: 18 42
Larger number = 42
```

Logic:

- The function returns whichever of the two values is bigger; main() prints it.

Remember:

- **A function can have more than one return statement**

Program 28: Factorial of a number using a function

Aim: To find n! = 1 x 2 x 3 x ... x n using a loop inside a function.

```
#include <stdio.h>

int factorial(int n);

int main()
{
    int a, result;
    printf("Enter a no.: ");
    scanf("%d", &a);

    result = factorial(a);
    printf("Factorial = %d\n", result);
    return 0;
}

int factorial(int n)
{
    int i, fact = 1;
    for (i = 1; i <= n; i++)
        fact = fact * i;
    return fact;
}
```

Sample Input / Output:

```
Enter a no.: 5
Factorial = 120
```

Logic:

- fact starts at 1 and is multiplied by i = 1, 2, ..., n.
- 5! = 1 x 2 x 3 x 4 x 5 = 120. By definition 0! = 1 (loop does not run, fact stays 1).

Remember:

- **Initialise fact = 1 (not 0)**
- **int is enough up to 12!**

Program 29: Sum of n natural numbers using a function

Aim: To validate input and return a sum from a function.

```
#include <stdio.h>

int calculateSum(int n);

int main()
{
    int n, sum;
    printf("Enter any natural no.: ");
    scanf("%d", &n);

    if (n < 1)
        printf("Given no. is invalid. Kindly enter a valid no.\n");
    else
    {
        sum = calculateSum(n);
        printf("The sum of first %d natural no. is %d\n", n, sum);
    }
    return 0;
}

int calculateSum(int n)
{
    int i, total = 0;
    for (i = 1; i <= n; i++)       /* i <= n so that n is also added */
        total += i;
    return total;
}
```

Sample Input / Output:

```
Enter any natural no.: 10
The sum of first 10 natural no. is 55
```

Logic:

- A natural number must be 1 or more, so n < 1 is rejected.
- total += i is the same as total = total + i.

Remember:

- **Loop condition is i <= n**
- **Initialise total = 0**

F. Number programs using loops (frequently asked)

Program 30: Reverse of a number

Aim: To reverse the digits of an integer using a while loop.

```
#include <stdio.h>

int main()
{
    int n, rev = 0;
    printf("Enter a no.: ");
    scanf("%d", &n);

    while (n > 0)
    {
        rev = rev * 10 + n % 10;    /* append last digit */
        n = n / 10;                 /* remove last digit */
    }
    printf("Rev is %d\n", rev);
    return 0;
}
```

Sample Input / Output:

```
Enter a no.: 1234
Rev is 4321
```

Logic:

- n % 10 gives the last digit; n / 10 removes it.
- Trace for 1234: rev = 4, 43, 432, 4321.

Remember:

- **rev = rev * 10 + n % 10;**
- **n = n / 10;**

Program 31: Fibonacci series (first n terms)

Aim: To print the series 0, 1, 1, 2, 3, 5, 8 ... using a for loop.

```
#include <stdio.h>

int main()
{
    int n, a = 0, b = 1, c, i;
    printf("Enter the number of terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series: ");
    for (i = 1; i <= n; i++)
    {
        printf("%d ", a);
        c = a + b;      /* next term = sum of previous two */
        a = b;
        b = c;
    }
    printf("\n");
    return 0;
}
```

Sample Input / Output:

```
Enter the number of terms: 7
Fibonacci Series: 0 1 1 2 3 5 8
```

Logic:

- Start with a = 0 and b = 1. Print a, then shift: a takes b, b takes the new sum c.

Remember:

- **c = a + b; a = b; b = c;**

Program 32: Palindrome number

Aim: To check whether a number reads the same forwards and backwards.

```
#include <stdio.h>

int main()
{
    int n, original, reverse = 0, remainder;
    printf("Enter a number: ");
    scanf("%d", &n);

    original = n;                       /* save the number */
    while (n != 0)
    {
        remainder = n % 10;
        reverse = reverse * 10 + remainder;
        n = n / 10;
    }

    if (original == reverse)
        printf("The no. is a palindrome\n");
    else
        printf("The no. is not a palindrome\n");

    return 0;
}
```

Sample Input / Output:

```
Enter a number: 121
The no. is a palindrome
```

Logic:

- Reverse the number and compare it with the saved original.
- 121 reversed is 121, so it is a palindrome; 123 reversed is 321, so it is not.

Remember:

- **Save original = n before the loop**

Program 33: Armstrong number

Aim: To check whether the sum of each digit raised to the power (number of digits) equals the number. Example: 153 = 1^3 + 5^3 + 3^3.

```
#include <stdio.h>

int main()
{
    int num, temp, r, digits = 0, power, i, sum = 0;
    printf("Enter any no. to check Armstrong: ");
    scanf("%d", &num);

    temp = num;
    while (temp != 0)           /* count the digits */
    {
        digits++;
        temp = temp / 10;
    }

    temp = num;
    while (temp != 0)
    {
        r = temp % 10;          /* last digit */
        power = 1;
        for (i = 1; i <= digits; i++)
            power = power * r;  /* r raised to 'digits' */
        sum = sum + power;
        temp = temp / 10;
    }

    if (sum == num)
        printf("%d is an Armstrong number\n", num);
    else
        printf("%d is not an Armstrong number\n", num);

    return 0;
}
```

Sample Input / Output:

```
Enter any no. to check Armstrong: 153
153 is an Armstrong number
```

Logic:

- Digits of 153: 3 -> 27, 5 -> 125, 1 -> 1. Sum = 153 which equals the number, so it is Armstrong.
- The first while loop counts the digits so the program works for any length.

Remember:

- **Keep a copy (temp) of num because num is needed for the final comparison**

Program 34: Sum of digits of a number

Aim: To add all the digits of an integer.

```
#include <stdio.h>

int main()
{
    int n, sum = 0, digit;
    printf("Enter a number: ");
    scanf("%d", &n);

    while (n != 0)
    {
        digit = n % 10;         /* extract last digit */
        sum = sum + digit;
        n = n / 10;             /* drop last digit */
    }
    printf("Sum of digits = %d\n", sum);
    return 0;
}
```

Sample Input / Output:

```
Enter a number: 4567
Sum of digits = 22
```

Logic:

- 4567: 7 + 6 + 5 + 4 = 22.

Remember:

- **Same pattern: % 10 then / 10**

Program 35: GCD (HCF) of two numbers - Euclid's method using a loop

Aim: To find the greatest common divisor.

```
#include <stdio.h>

int main()
{
    int a, b, temp;
    printf("Enter two no.: ");
    scanf("%d %d", &a, &b);

    while (b != 0)
    {
        temp = b;
        b = a % b;      /* remainder becomes the new divisor */
        a = temp;
    }
    printf("GCD = %d\n", a);
    return 0;
}
```

Sample Input / Output:

```
Enter two no.: 48 18
GCD = 6
```

Logic:

- Repeat: (a, b) -> (b, a % b) until b becomes 0. The remaining a is the GCD.
- 48,18 -> 18,12 -> 12,6 -> 6,0, so GCD = 6.

Remember:

- **while (b != 0)**
- **Answer is stored in a**

G. Recursion programs

Program 36: Recursion basic example - print Hello n times

Aim: To show base case and recursive case.

```
#include <stdio.h>

void printHello(int n)
{
    if (n == 0)                 /* base case: stop */
        return;
    printf("Hello\n");
    printHello(n - 1);          /* recursive case: smaller input */
}

int main()
{
    printHello(3);
    return 0;
}
```

Output:

```
Hello
Hello
Hello
```

Logic:

- Each call prints Hello and calls itself with n - 1 until n becomes 0 (base case).

Remember:

- **Without a base case the recursion never stops**

Program 37: Factorial using recursion

Aim: n! = n x (n-1)! with base case 0! = 1.

```
#include <stdio.h>

int fact(int n)
{
    if (n == 0)
        return 1;                   /* base case */
    else
        return n * fact(n - 1);     /* recursive case */
}

int main()
{
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    printf("Factorial = %d\n", fact(num));
    return 0;
}
```

Sample Input / Output:

```
Enter a number: 4
Factorial = 24
```

Logic:

- fact(4) = 4 x fact(3) = 4 x 3 x fact(2) = 4 x 3 x 2 x fact(1) = 4 x 3 x 2 x 1 x fact(0).
- fact(0) returns 1, then values return one by one: 1, 1, 2, 6, 24.

Remember:

- **Base case: n == 0 returns 1**

Program 38: Fibonacci series using recursion

Aim: fib(n) = fib(n-1) + fib(n-2) with base cases fib(0) = 0 and fib(1) = 1.

```
#include <stdio.h>

int fibonacci(int n)
{
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main()
{
    int i, n;
    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++)
        printf("%d ", fibonacci(i));

    printf("\n");
    return 0;
}
```

Sample Input / Output:

```
Enter number of terms: 7
0 1 1 2 3 5 8
```

Logic:

- Each call makes two recursive calls, so it forms a tree (tree recursion).
- fib(5) = fib(4) + fib(3) = 3 + 2 = 5.

Remember:

- **Two base cases: 0 and 1**

Program 39: GCD (greatest common divisor) using recursion

Aim: gcd(a, b) = gcd(b, a % b) with base case gcd(a, 0) = a.

```
#include <stdio.h>

int gcd(int a, int b)
{
    if (b == 0)
        return a;                   /* base case */
    else
        return gcd(b, a % b);       /* recursive case */
}

int main()
{
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("GCD = %d\n", gcd(a, b));
    return 0;
}
```

Sample Input / Output:

```
Enter two numbers: 48 18
GCD = 6
```

Logic:

- gcd(48,18) -> gcd(18,12) -> gcd(12,6) -> gcd(6,0) = 6.
- Answer 6 is returned back through every call.

Remember:

- **Base case: b == 0 returns a**

Program 40: Sum of digits using recursion

Aim: sumDigits(n) = last digit + sumDigits(n / 10) with base case n == 0.

```
#include <stdio.h>

int sumDigits(int n)
{
    if (n == 0)
        return 0;                           /* base case */
    return (n % 10) + sumDigits(n / 10);    /* last digit + rest */
}

int main()
{
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);

    printf("Sum of digits = %d\n", sumDigits(n));
    return 0;
}
```

Sample Input / Output:

```
Enter a number: 4567
Sum of digits = 22
```

Logic:

- sumDigits(4567) = 7 + sumDigits(456) = 7 + 6 + sumDigits(45) = ... = 22.

Remember:

- **n % 10 = last digit, n / 10 = remaining number**

Program 41: Indirect recursion (two functions call each other)

Aim: To show that a function can call itself through another function.

```
#include <stdio.h>

void funB(int n);                   /* prototype is needed */

void funA(int n)
{
    if (n > 0)
    {
        printf("A%d ", n);
        funB(n - 1);                /* A calls B */
    }
}

void funB(int n)
{
    if (n > 0)
    {
        printf("B%d ", n);
        funA(n - 1);                /* B calls A */
    }
}

int main()
{
    funA(4);
    printf("\n");
    return 0;
}
```

Output:

```
A4 B3 A2 B1
```

Logic:

- funA -> funB -> funA -> ... The chain stops when n reaches 0 (base case).

Remember:

- **Indirect recursion needs a prototype for the function defined later**

PART 3 - IMPORTANT SYNTAX AND DIFFERENCES

A. Syntax Cheat-Sheet

```
if:                              if-else:
  if (condition)                   if (condition)
  {                                {   statements;   }
      statements;                  else
  }                                {   statements;   }

else-if ladder:                  conditional operator:
  if (c1)       { ... }            var = (condition) ? value1 : value2;
  else if (c2)  { ... }
  else          { ... }

for:                             while:
  for (init; condition; update)    while (condition)
  {   statements;   }              {   statements;   }

do-while:                        break / continue:
  do                               if (condition) break;      // exit loop
  {   statements;   }              if (condition) continue;   // next iteration
  while (condition);

switch:                          goto:
  switch (int_or_char_expr)        goto label;
  {                                ...
    case 1: ...; break;            label:
    case 2: ...; break;                statement;
    default: ...;
  }

function:                        recursion:
  int add(int, int);   // proto     int fact(int n)
  s = add(a, b);       // call      {
  int add(int x, int y)                if (n == 0) return 1;   // base
  {   return x + y;   }                return n * fact(n-1);   // recursive
                                   }
```

Item	Details
Format specifiers	`%d` int, `%f` float, `%c` char. Use `&` in scanf (e.g. `scanf("%d", &n);`) but not in printf.
Relational operators	`== != > < >= <=` (== compares, = assigns)
Logical operators	`&&` (AND), `||` (OR), `!` (NOT)
Arithmetic	`+ - * / %` (% = remainder, only for integers; n % 10 = last digit, n / 10 = remove last digit)
Increment	`i++` adds 1; `sum += i` means `sum = sum + i`
Print a % sign	`printf("%%");`

B. Difference Tables

1. if vs if-else

Point	if	if-else
Meaning	Block executes only when the condition is true	One block executes if true, another block executes if false
Condition false	Nothing happens, control goes to next statement	else block executes
Keywords	if	if and else
Example	`if (n > 0) printf("+ve");`	`if (n % 2 == 0) ... else ...`

2. while vs do-while

Point	while	do-while
Type	Entry-controlled loop	Exit-controlled loop
Condition checked	Before the body	After the body
Minimum executions	0 (may never run)	1 (runs at least once)

Point	while	do-while
Syntax end	`while (cond) { ... }` no semicolon	`do { ... } while (cond);` semicolon compulsory

3. for vs while vs do-while

Point	for	while	do-while
Best used when	Number of iterations is known	Number of iterations unknown; condition-based	Body must run at least once
Initialisation / update	Both written in the for statement	Initialise before, update inside the body	Initialise before, update inside the body
Condition check	Before body	Before body	After body
Loop type	Entry-controlled	Entry-controlled	Exit-controlled

4. break vs continue

Point	break	continue
Action	Terminates the loop (or switch) completely	Skips the current iteration only
Control goes to	The statement after the loop	Next iteration (condition / update of the loop)
Used in	Loops and switch	Loops only
Example	`if (i == 7) break;`	`if (i == 7) continue;`

5. switch vs if-else ladder

Point	switch	if-else ladder
Expression type	int or char only	Any condition: int, float, ranges, logical expressions
Test	Equality with constant case labels	Any relational / logical expression
Default	default case (optional)	Last else part
break	Needed after each case to avoid fall-through	Not needed
Best for	Menu / fixed choices (calculator, days)	Ranges (grades, percentage)

6. Predefined vs user-defined function

Point	Predefined / library function	User-defined function
Defined by	Compiler library	Programmer
Header file	Required (stdio.h, math.h)	Not required
Examples	printf(), scanf(), sqrt()	addNumbers(), factorial()

7. Function declaration vs definition vs call

Point	Declaration (prototype)	Definition	Call
Purpose	Tells compiler name, return type, parameter types	Contains the actual body of code	Executes the function
Position	Top, above main()	Usually below main()	Inside main() or another function
Ends with	Semicolon	Block { }	Semicolon
Example	`int add(int, int);`	`int add(int a,int b){...}`	`s = add(x, y);`

8. Iteration (loop) vs recursion

Point	Iteration	Recursion
Repetition by	Loop statements (for, while, do-while)	Function calling itself
Stops when	Loop condition becomes false	Base case is reached
Memory	Less	More (each call is pushed on the call stack)
Code size	Usually longer	Short and simple for problems like factorial, Fibonacci, GCD
Risk	Infinite loop if update is missing	Infinite calls if base case is missing

9. Direct vs indirect vs tree recursion

Type	Calls	Example
Direct	Function calls itself once per call	Factorial
Indirect	A calls B, B calls A	funA() and funB()
Tree	Function calls itself two or more times in one call	Fibonacci

10. = (assignment) vs == (comparison)

Operator	Meaning	Example
=	Stores a value in a variable	`x = 5;` (x becomes 5)
==	Checks whether two values are equal (result true / false)	`if (x == 5)`

PART 4 - SOLVED OUTPUT-BASED QUESTIONS

Predict the output. Answers were obtained by compiling and running each program.

Q1. if-else with a statement after it

```
#include <stdio.h>
int main()
{
    int a = 5;
    if (a > 3)
        printf("A");
    else
        printf("B");
    printf("C");
    return 0;
}
```

Output:

```
AC
```

Explanation: a > 3 is true so A is printed; else is skipped; printf("C") is outside the if-else and always runs.

Q2. Semicolon after if

```
#include <stdio.h>
int main()
{
    int x = 10;
    if (x > 20);
        printf("Hello");
    return 0;
}
```

Output:

```
Hello
```

Explanation: The semicolon makes an empty if-body. printf is a separate statement, so Hello is printed even though the condition is false.

Q3. = instead of ==

```
#include <stdio.h>
int main()
{
    int x = 0;
    if (x = 5)
        printf("True");
    else
        printf("False");
    return 0;
}
```

Output:

```
True
```

Explanation: x = 5 is an assignment. It stores 5 in x and the value of the expression is 5 (non-zero = true).

Q4. switch without break (fall-through)

```
#include <stdio.h>
int main()
{
    int a = 2;
    switch (a)
    {
        case 1: printf("One ");
        case 2: printf("Two ");
        case 3: printf("Three ");
        default: printf("Default");
    }
    return 0;
}
```

Output:

```
Two Three Default
```

Explanation: case 2 matches, and since there is no break, every statement after it also executes.

Q5. break inside a loop

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
            break;
        printf("%d ", i);
    }
    return 0;
}
```

Output:

```
1 2
```

Explanation: When i becomes 3 the loop is terminated immediately.

Q6. continue inside a loop

```
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 5; i++)
    {
        if (i == 3)
            continue;
        printf("%d ", i);
    }
    return 0;
}
```

Output:

```
1 2 4 5
```

Explanation: When i is 3 the printf is skipped and the loop moves to the next iteration.

Q7. do-while with a false condition

```
#include <stdio.h>
int main()
{
    int i = 11;
    do
    {
        printf("i = %d\n", i);
        i++;
    }
    while (i <= 10);
    return 0;
}
```

Output:

```
i = 11
```

Explanation: The body executes once before the condition (12 <= 10) is tested.

Q8. while loop

```
#include <stdio.h>
int main()
{
    int i = 1;
    while (i <= 3)
    {
        printf("%d ", i);
        i++;
    }
    printf("\ni after loop = %d", i);
    return 0;
}
```

Output:

```
1 2 3
i after loop = 4
```

Explanation: The loop ends when i becomes 4; that value is still in i after the loop.

Q9. Nested if tracing

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20, c = 5;
    if (a > b)
    {
        if (a > c)
            printf("a is greatest");
        else
            printf("c is greatest");
    }
    else
    {
        if (b > c)
            printf("b is greatest");
        else
            printf("c is greatest");
    }
    return 0;
}
```

Output:

```
b is greatest
```

Explanation: a > b is false, so the else block runs; there b > c is true.

Q10. Recursion - sum of 1 to n

```
#include <stdio.h>
int f(int n)
{
    if (n == 0)
        return 0;
    return n + f(n - 1);
}
int main()
{
    printf("%d", f(4));
    return 0;
}
```

Output:

```
10
```

Explanation: f(4) = 4 + f(3) = 4 + 3 + f(2) = 4 + 3 + 2 + f(1) = 4+3+2+1+f(0) = 10.

PART 5 - REVISION

A. Practice Questions from Your Notes - Where to Find the Solution

Question	Program
Accept a number and print if it is odd	Program 1
Accept two floating numbers, compute ratio, exchange if ratio > 0	Program 2
Check two numbers equal / unequal; character equal to y -> Yes else No	Programs 3, 4
Check whether a number is odd or even	Program 5
Check larger and smaller of two numbers	Program 6
Check number positive, negative (or zero)	Program 7
Check a number is divisible by 5	Program 8
Check whether a year is leap	Program 9
Check a character is upper case or lower case alphabet	Program 10
Input an alphabet and check vowel or consonant	Program 11
Largest among 3 numbers (nested if); largest / smallest among 3 or 4 numbers	Programs 12, 13
Result of a student, 5 subjects, percentage and grade (A to E)	Program 14
Sum of first 10 natural numbers	Program 17
Sum of first 10 numbers - loop stops on negative number (break)	Program 21
Sum of first 10 numbers - negative number not added (continue)	Program 22
Switch statement / calculator	Programs 23, 24
goto statement (even / odd)	Program 25
Add two numbers, larger number, factorial, sum of n natural numbers using functions	Programs 26 to 29
Reverse, Fibonacci, Palindrome, Armstrong, Sum of digits, GCD	Programs 30 to 35
Recursion: factorial, Fibonacci, GCD, sum of digits	Programs 37 to 40

B. Short Questions and Answers (Viva / 2-3 marks)

Q1. What are decision-making statements?

Ans: Statements that execute a block only if a condition is true. C provides if, switch, conditional operator (?:) and goto.

Q2. What is the difference between if and if-else?

Ans: if executes a block only when the condition is true and does nothing when false; if-else executes the else block when the condition is false.

Q3. What is a nested if?

Ans: An if or if-else statement written inside another if-block or else-block.

Q4. What is an else-if ladder?

Ans: A chain of conditions tested in sequence. The first true condition's block runs and control leaves the ladder; if none is true the final else runs.

Q5. What is a loop? Name the loops in C.

Ans: A loop repeats a block of code until a condition is false. Loops: for, while, do-while.

Q6. Why is do-while called an exit-controlled loop?

Ans: Because the condition is tested after the body, so the body executes at least once.

Q7. What is the use of break and continue?

Ans: break ends the loop (or switch) immediately; continue skips the current iteration and continues with the next one.

Q8. What happens if break is not used in a switch?

Ans: Fall-through: all statements after the matching case label also execute until a break or the end of the switch.

Q9. Which data types can be used in a switch expression?

Ans: Only int and char.

Q10. What is goto and where is it used?

Ans: An unconditional jump to a labelled statement in the same function; used for error handling and exiting deeply nested loops.

Q11. What is a function? Why do we need it?

Ans: A set of statements that performs a specific task. It improves readability and reusability, makes debugging easier and reduces code size.

Q12. What is a function prototype?

Ans: The declaration that tells the compiler the function name, return type and parameter types before use; placed above main().

Q13. Name the two categories of functions.

Ans: Predefined (library) functions like printf(), scanf(), sqrt(); and user-defined functions.

Q14. What is the return type void?

Ans: It means the function returns no value.

Q15. What is recursion? What are base case and recursive case?

Ans: A function calling itself. Base case stops the recursion; recursive case calls the function again with a smaller input.

Q16. How does recursion use memory?

Ans: Each call is pushed on the call stack; after the base case the calls return one by one from top to bottom.

Q17. Name the types of recursion.

Ans: Direct, indirect and tree recursion.

C. Common Mistakes to Avoid

Mistake	Correct way
`if (x > 5);` - semicolon after condition	`if (x > 5)` with no semicolon (else the body is empty)
`if (x = 5)` - assignment in condition	`if (x == 5)`
Missing `&` in scanf	`scanf("%d", &n);`
Missing `break;` in switch case	Add break after every case (unless fall-through is wanted)
Missing semicolon after `do { } while (c)`	`while (condition);` is compulsory
Loop variable never updated -> infinite loop	Update inside the body or in the for statement
`sum` / `fact` not initialised	`sum = 0;` for sums, `fact = 1;` for factorial
Loop `i < n` when n must also be included	`i <= n`
Recursion without a base case	Always write the stopping condition first
Integer division `total / 500` for percentage	`total / 500.0 * 100`
Header file missing (`#include <stdio.h>`)	Include the header of every library function used

D. Last-Minute Revision Checklist

Tick these before the exam

• Decision statements: if, switch, ?:, goto - and the 4 loops/jumps classification (conditional, looping, jump).
• Structure of C program: 6 sections in order (documentation, preprocessor, definition, global declaration, main, sub-programs).
• Write syntax and flow chart for: if, if-else, nested if, else-if ladder, for, while, do-while, switch.
• for: initialisation once -> test -> body -> update -> test ... ; while: test first; do-while: body first.
• break only in loop / switch; continue only in loops; switch works with int / char only; default is optional.
• Functions: 3 parts (declaration, definition, call), return type, void, predefined vs user-defined, why we need functions.
• Recursion: base case, recursive case, call stack, 3 types; dry run of fact(4), fib(5), gcd(48,18).
• Learn by heart: odd/even, leap year, largest of 3, grade ladder, reverse, palindrome, Armstrong, sum of digits, Fibonacci, factorial, GCD (loop + recursion).
• Digit patterns: n % 10 = last digit, n / 10 = remove last digit.